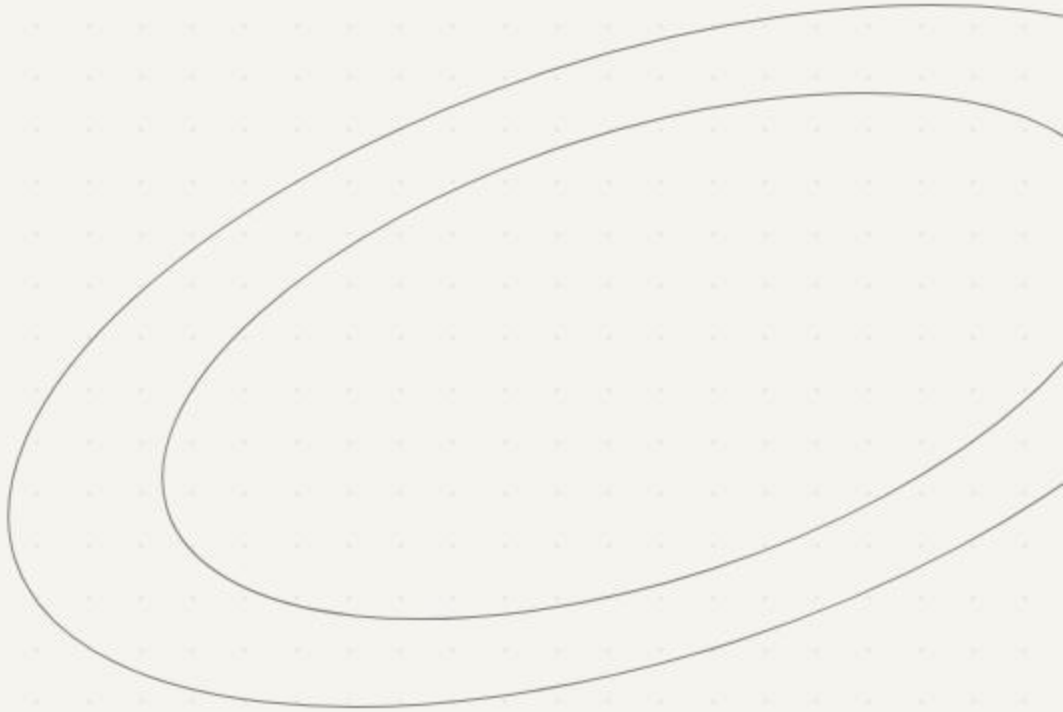
CELESTIAL MECHANICS × AI · 2026

From the next token to the next theorem

How a model works — and how I use it in practice



Bowen Liu

What the model sees: ten integers

EN · ONE SENTENCE

Celestial mechanics is the poetry of orbits.

Cel	estial	_mechanics	_is	_the	_poetry	_of	_or	bits	.
69126	90010	52347	382	290	38153	328	503	24043	13

46 characters → **10** tokens → 10 integers · _ marks a leading space

CHINESE · FOUNDATIONS OF CELESTIAL MECHANICS

本书系统地介绍了天体力学

本	书	系统	地	介绍	了	天	体	力	学
4087	20121	31936	5557	37887	4531	867	7074	11343	8590

12 characters → **10** tokens → 10 integers · two-character words often map to one token

“This book systematically introduces celestial mechanics.”

01 Where do these integers come from?

02 How does it predict the next one?

03 How do we put it to work?

Tokenization is table lookup

Three-body problem is a classic problem in celestial mechanics.

Three	-body	problem	_is	_a	_classic	_problem	_in	_celestial	_mechanics	.
25843	19146	4792	382	261	13686	4792	306	134437	52347	13

_ marks a leading space · Celestial → Cel + estial

≈5
characters / English word
including the space, typical

≈4
characters / token
English rule of thumb

≈1.3
tokens / English word
≈ 5 ÷ 4

50k–200k
typical vocabulary size
50k – 200k

WHY Characters: little information, 4–5× longer → Words: vocabulary explodes (millions) → **Subwords: the compromise**

GPT-2 50k · GPT-4 100k · Llama 3 128k · GPT-4o 200k

> Subwords are an engineering compromise between **sequence length** and **vocabulary size**.

English: 1 token ≈ 4 characters ≈ 0.75 words

The vocabulary: a fixed lookup table

Foundations of Celestial Mechanics · row number = ID · sorted by ID

ID	TOKEN	
290	_the	
2392	_book	
2500	This	
18864	_fundamental	
23753	_concepts	
52347	_mechanics	
134437	_celestial	← query
199999	<|endoftext|>	special

8 rows shown · 200,019 rows in total

Vocabulary size = number of rows

200,019

o200k · GPT-4o generation

- **0**
- **50k** GPT-2
- **100k** GPT-4 gen
- **128k** Llama 3
- **200k** GPT-4o gen

200,019 × 4,096 ≈ **820M parameters**

> An ID is a **row number**; changing models means changing the table.

Tokens: the model's unit and the billing unit



Last month · ≈110k tokens per request · ≈¥0.01 each

≈ 0.3 token
per English character

≈ 0.6 token
per Chinese character

Rule of thumb: a word, a number or a symbol ≈ 1 token; tokenizers differ, so trust the usage the API returns.

> Token is both the smallest unit of the model and the smallest unit of the bill.

Where the money goes on a single call

价格[2]	百万tokens输入（缓存命中）	空闲时段	0.02元
		高峰时段	0.04元
	百万tokens输入（缓存未命中）	空闲时段	1元
		高峰时段	2元
	百万tokens输出	空闲时段	4元
		高峰时段	8元

deepseek-flash · 1M context · 384K output · CNY per million tokens

1M context ≈ the whole Three-Body trilogy

One call · off-peak

Input 100k + output 2k

¥0.01 cache hit

¥0.11 cache miss

> The same context, cached or not, differs by an **order of magnitude**.

Text becomes an ID, then a vector

PIECE

`problem`

A unit after tokenization

→

ID

4792

Row 4792 of the vocabulary

→

VECTOR

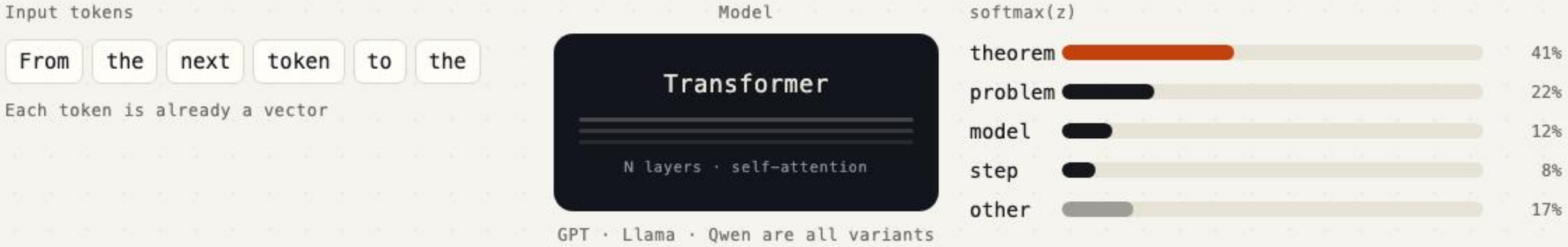
`0.12 −0.44 0.08 …`

4,096 dimensions · learned

> The ID is an **address**; the vector is the **content**.

This step is called embedding; no dimension has a name, yet distance carries meaning.

What the model does: input → probability distribution



$$z = f_\theta(x_{1:t}) \in \mathbb{R}^{|V|} \qquad p_\theta(y \mid x_{1:t}) = \frac{\exp(z_y)}{\sum_{v \in V} \exp(z_v)}$$

$|V| = 200{,}019$

$\Sigma_y\, p_\theta(y \mid x_{1:t}) = 1$

> A scorer: it gives every token in the vocabulary a **score**, then turns scores into **probabilities**.

The model is stateless

CALL A

```
system    You are a celestial mechanics assistant
user      What is the Hill region?
assistant The Hill region is …
user      Why does it matter?
assistant Because it defines …
```

CALL B · A FRESH CALL

```
system    You are a celestial mechanics assistant
user      Why does it matter?
assistant ("it" has no referent here)
```

> The model is stateless; a "conversation" is the client **resending** the history.

Parameters stay fixed at inference time; state lives only in the context.

Training: pretraining and post-training

STAGE 01

Pretraining

Large-scale text, code and other data

Learn language, knowledge and patterns

`Output: a base model`

→

STAGE 02

Post-training

SFT · SUPERVISED FINE-TUNING

Learn by example

Shape responses and task behavior.

RL · OUR FOCUS

Learn from reward

Try, evaluate, improve the policy.

> Our focus is **RL**: improving **problem-solving** through reward.

RL: actions, outcomes and feedback

Act in an environment; use outcome feedback to improve action choices.

Observe environment → Take action → Outcome → Feedback / reward → Improve policy ↺

ENVIRONMENT

Produce an outcome

Actions affect the environment; new observations return.

FEEDBACK

Evaluate the result

Reward expresses progress or success.

POLICY

Improve action choices

Use experience to seek higher long-run return.

> RL uses **outcome feedback** to learn **how to choose actions**.

Reference: Sutton & Barto, Reinforcement Learning: An Introduction.

Games made reinforcement learning visible

2015

Atari · DQN

Pixels → actions

Reward: game score

2016

AlphaGo

Self-play + search

Reward: winning

2019

OpenAI Five

Dota 2 team play

Reward: match outcome

2019

AlphaStar

StarCraft II strategy

Reward: match outcome

- Explicit rules
- Measurable outcomes
- Many repeatable trials

> The game engine supplies an **environment and feedback**.

Sources: Mnih et al., 2015 · Silver et al., 2016 · OpenAI Five, 2019 · Vinyals et al., 2019.

A score can teach a strategy

Breakout: move the paddle, hit the ball, clear the bricks.

TRY

Different moves

Explore timing and direction.

OBSERVE

Score the outcome

Miss the ball, or clear bricks.

LEARN

Better strategies

Learn actions with delayed benefits.

> Repeated trials + outcome feedback → a policy that seeks higher reward.

> A useful environment lets the agent **try, evaluate and try again**.

Example: Mnih et al., Nature (2015), Breakout demonstration.

Math and code share a feedback advantage

01 · DEFINED ENVIRONMENT

Rules & assumptions

An interface, a specification, a theorem.

02 · EXPLICIT TARGET

A checkable goal

Expected behavior or a precise claim.

03 · USEFUL FEEDBACK

Tests & counterexamples

Reject failures and compare candidates.

Two levers: better policies + more compute for candidate generation and verification.

RL improves problem-solving policies; compute expands **verifiable search**.

Finite search space + a solution exists + reliable, decidable checks → exhaustive search can find it; cost may be high.

Code and math exercise shared reasoning

CODE PRACTICE

- Track variables and state
- Respect types and constraints
- Decompose and debug

MATH PRACTICE

- State assumptions explicitly
- Maintain symbolic consistency
- Build and check multi-step arguments

Reported transfer: math RL → coding scores · code training → state tracking.

Shared skills support **transfer**; the data mixture determines the gain.

Sources: Chen et al., AceReason (2025) · Kim et al. (2024) · Shao et al., DeepSeekMath (2024).

A test or counterexample changes the next step

Illustrative tasks from Foundations of Celestial Mechanics.

CODE · AN ORBIT ROUTINE

- Test a circular orbit
- Test high eccentricity
- Check step-size convergence

MATH · A PROPOSED CLAIM

“Every Kepler orbit is bounded.”

Counterexample: a hyperbolic orbit.

```
The claim needs stronger assumptions.
```

Candidate → Check → Feedback → Next attempt

> Checks turn a proposed answer into **actionable feedback**.

Coding gives an agent a place to act

Read files → Edit → Run tests → Inspect & revise ↻

WORKSPACE

Files and specifications define the task.

TOOLS

Changes can be executed immediately.

FEEDBACK

Failures guide the next edit.

Examples: Claude Code, Codex, OpenCode and other coding agents.

> Code connects **generation, action and feedback** in one workspace.

Earlier generate-and-filter example: Li et al., AlphaCode, Science (2022).

Four tasks, different feedback conditions

Code, math, product design, trading: how do goals, environments and feedback fit?

01 · CODE

Executable checks

- Target: specified behavior
- Feedback: compiler, tests, errors
- Trials: run and revise locally

02 · MATH

Explicit assumptions & goals

- Target: solve or prove under assumptions
- Feedback: answers, proof checks, counterexamples
- Trials: generate and verify candidates

03 · PRODUCT DESIGN

Long real-world feedback loops

- Target: user value and differentiation
- Feedback: slower, plural, empirical
- Judgment: distinctive, sometimes non-consensus

04 · TRADING

Clear reward, changing markets

- Target: profit, risk and cost
- Environment: incomplete info, shifting markets
- Validation: backtests versus future returns

> Checkable goals turn more compute into **systematic search and candidate filtering**.

For finite, solvable tasks with reliable, decidable checks, exhaustive search finds a valid solution; cost still matters.

Correctness is a floor; quality is an ambition

	CODE	MATHEMATICS
CHECKABLE STRUCTURE	Syntax, types, tests, specifications	**Definitions, assumptions, proof steps**
CORRECTNESS	Specified behavior is satisfied	**A claim follows from its assumptions**
QUALITY	Clarity, maintainability, efficiency	**Elegance, generality, usefulness**

Structured artifacts make checks easier; data quality still requires curation.

> Both communities pursue **good work** beyond a pass/fail result.

Learning and working use different loops

RL TRAINING

Improve the policy

Try → reward → update parameters

AGENT WORKFLOW

Improve the result

Try → feedback → revise the work

> A coding agent combines a model with context, tools, checks and a control loop.

> Now: how I organize the **working loop**.

From talking to working

TALKING · 2023

ChatGPT

Instruction tuning + RLHF

prompt / system

THINKING

CoT

Write the intermediate steps

context engineering · pruning

WORKING

Agent

tool calling · loop

memory

> Each step solves the problem the previous step left open.

2023: from completion to conversation

user	Explain the Hill region in one sentence.
assistant	The region of space a body can reach at a given energy.

INSTRUCTION TUNING

Learns the dialogue format: respond to the request

RLHF

Learns answers people prefer: helpful, harmless

SCOPE

Text only: no tools, no memory, no verification

> Conversation is a trained behavior.

Prompt: change the distribution with content

SYSTEM · ROLE
You are a celestial mechanics teaching assistant.

CONSTRAINTS
Write the formula first, then substitute, then give the number.

USER · TASK
A period of 2 years; the semi-major axis grows 4×. What is the new period?

FEW-SHOT · EXAMPLES
a × 9 → T × 27
a × 16 → T × 64



$$p_\theta(y \mid x) \rightarrow p_\theta(y \mid x, c, e)$$

p(16 | x) = 0.18
p(16 | x, c, e) = 0.71

> Adding content is how you change the **conditional probability**.

CoT: write the thinking down

Scope: verbal and logical reasoning; arithmetic is a separate step.

Question

A period of 2 years; the semi-major axis grows 4×. What is the new period?

DIRECT ANSWER

8 years ✗ — a number without a plan

COT · PLAN

$T \propto a^{3/2}$; $T = 2 \times 4^{3/2}$

plan ✓ · arithmetic next

TOOL · COMPUTE

```
run_python("2 * 4**1.5") → 16.0 ✓
```

> CoT writes the plan; the tool computes the number.

WHAT COT DOES

Turns words into a plan: parse, recall, set up the relation

WHAT REMAINS

Arithmetic: a separate, tool-friendly step

ESSENCE

Explicit steps; exact numbers are a separate skill

The inflation of the context window



COST

Billed per token; long prefills are slower

LATENCY

Prefill and attention costs rise

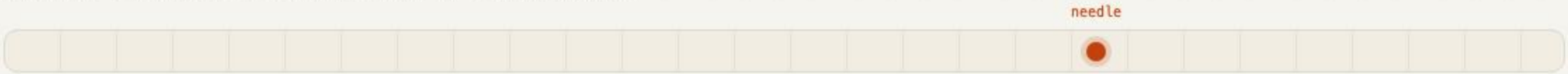
DILUTION

Middle content gets ignored; signal drowns in noise

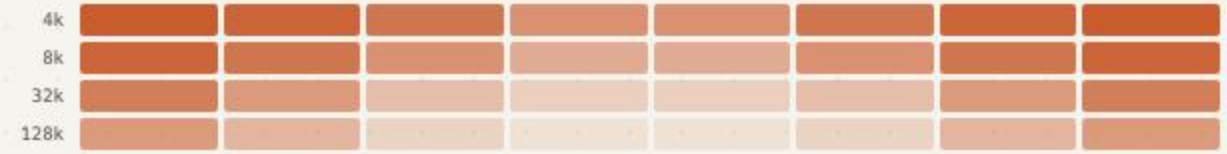
> A longer window raises the ceiling; relevance decides what lands.

Context is finite

Needle in a haystack: hide one sentence in a long document



Retrieval accuracy (illustrative)



Horizontal axis: needle position 0 → 100%

METHOD

Hide a sentence, vary length and position, ask what it was

OBSERVED FAILURE MODE

Length and position can affect retrieval; behavior varies by model and task

IMPLICATION

A big window is a budget; retrieval still matters

Schematic, not measured data · See Liu et al., Lost in the Middle (2023).

> Window capacity and **effective use of context** are different measures.

Context engineering: the window is a workbench

What goes in the window

```
system: rules
history: recent turns
retrieval: relevant documents
tools: returned results
current: this question
```

TOO LONG

Retrieve; keep only what is relevant

TOO SCATTERED

Structure it; label the blocks

LOST IN THE MIDDLE

Put key facts at the start and end

OVERFLOW

Compress, summarize, sub-agents

> Relevance beats length.

CoT pruning: keep the thoughts that matter

A long chain of thought

Read problem → Name variables → Expand → ~~Repeat~~ → Case split → ~~Side branch~~ → Main line → Substitute → Check → Answer

PRUNE

Drop repeated and irrelevant branches

DISTILL

Teach a short CoT from a long one

SUMMARIZE

Compress a long chain into a short one

BUDGET

Cap thinking tokens; stop early

> Thinking has a budget too.

Tool calling: let a program do the arithmetic

```
Tool: run_python(code: str) → str

call   run_python("print(2 * 4**1.5)")
obs    16.0

// T = 2 × 4**1.5 = 16
```

01 DEFINE

Tool signature: name, arguments, return

02 CALL

The model emits a snippet

03 EXECUTE

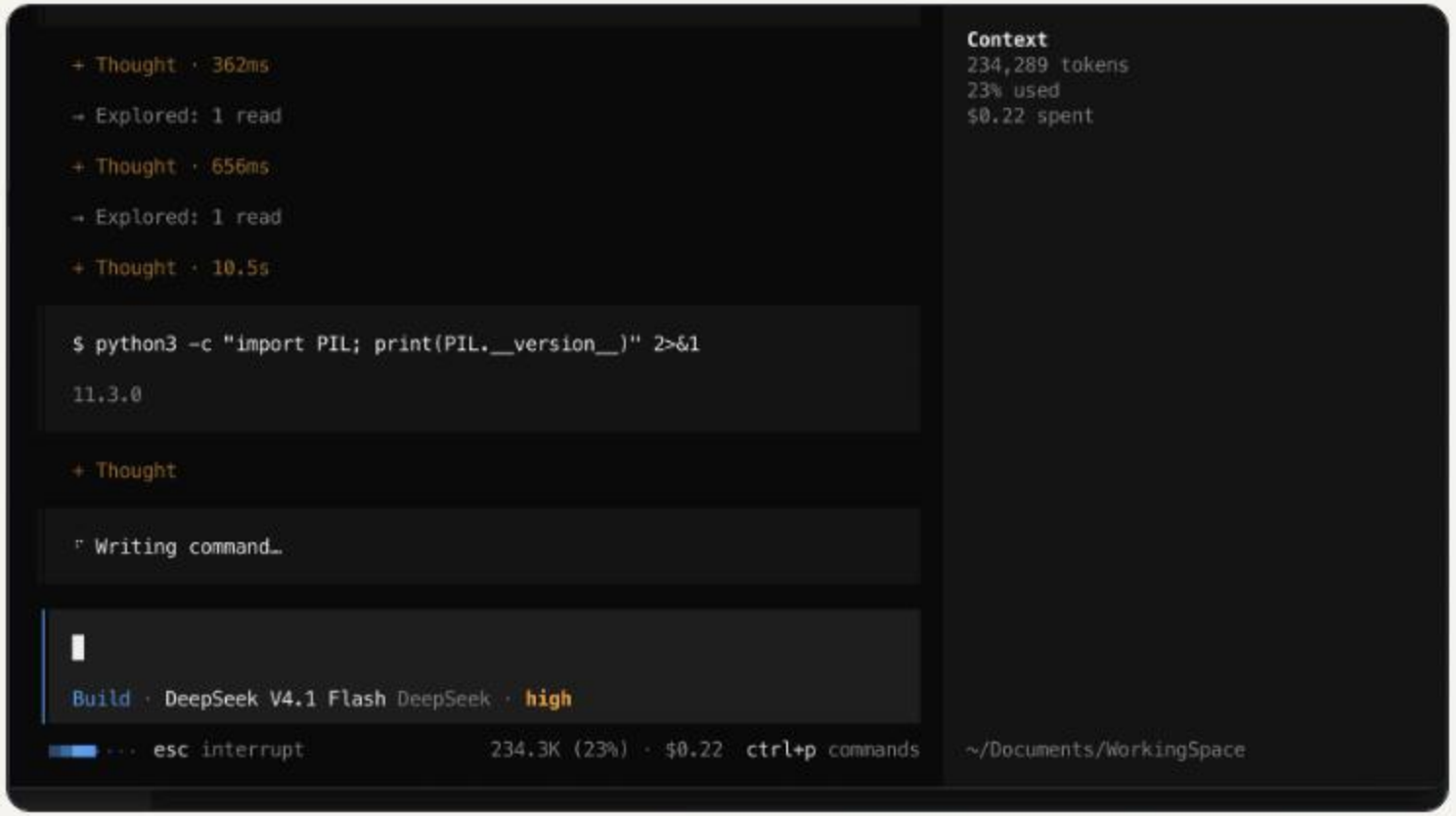
It really runs, in a sandbox

04 OBSERVE

The result returns to the context; continue

> The model proposes; the program computes.

Agent: the model inside a loop



01
Loop
Thought → tool call → observation, repeat

02
Context
234K tokens · 23% used · $0.22

03
Client
Model + tools + session + billing

04
Model
DeepSeek V4.1 Flash · high

> The model proposes; the client turns proposals into **actions**.

Memory: signal and noise

What is in the window

Signal
Task · constraints · decisions · verified facts

Noise
Chat · repetition · stale info · raw logs

Write
Store decisions and facts only

Retrieve
Fetch for the current task

Compress
Summarize, merge

Forget
Delete when stale

WORKING MEMORY

The current window; only what this step needs

EPISODIC MEMORY

Past summaries; decisions and open questions

SEMANTIC MEMORY

External store + retrieval; fetch on demand

PROCEDURAL MEMORY

system + tools; rules and capabilities

> Memory works by selection: store little, retrieve precisely.

When code outruns human review

Claude Code's review problem: code scales faster than human attention.

SCALE PRESSURE · ANTHROPIC MONOREPO

2,500 → 30,000+

Changes landing per day · May → Sep 2026

Claude Code has a separate repository.

PRODUCTION

Agents write more code, faster.

BOTTLENECK

Human review time stays limited.

CONSEQUENCE

Line-by-line review cannot keep up.

Goal: reduce **review + repair time**, with reverts and incidents as guardrails.

> The next gain comes from **redesigning review**.

Source: Osmani, Lunt & Wang · 15 Oct 2026 · How the Claude Code team does code review · Team-reported practice.

Move routine review into the system

Checks throughout the workflow; human attention goes where it matters.

01 · BEFORE SUBMISSION

Produce evidence

- Run tests, build, inspect UI
- Fresh-context agent review
- Fix before opening review

02 · DURING REVIEW

Layer the checks

- AI review; verify findings
- Security & automated tests
- Resolve every finding

03 · APPROVAL & RELEASE

Control the risk

- Risk-based human approval
- Merge checks; staged release
- Detect fast; roll back fast

Shared context: CLAUDE.md = project conventions · REVIEW.md = local failure criteria + evidence.

> Routine checks run continuously; people focus on **judgment and risk**.

Source: Osmani, Lunt & Wang · 15 Oct 2026 · How the Claude Code team does code review · Team-reported practice.

Human attention follows the risk

The team's approval policy, with explicit ownership.

CHANGE RISK	EXAMPLES	HUMAN APPROVALS
LOW	Docs, copy, internal-only features	**0 required***
MEDIUM	Ordinary product changes	**1 person***
HIGH	Highest-risk code; designated paths	**2 people**

* Lighter paths: trusted authors only, after required checks. Sensitive paths may require extra approval.

DIRECTION

A person decides what to build.

EVIDENCE

Approval covers a specific commit.

ACCOUNTABILITY

The directing person owns the change.

> Required human approval stays **human**; responsibility stays **explicit**.

Source: Osmani, Lunt & Wang · 15 Oct 2026 · How the Claude Code team does code review · Team-reported practice.

Make review improve — and stop

Past failures become evals; review loops get escalation rules.

Incident / revert → Engineer-confirmed case → Daily eval → Reviewer update ↺

MEASURE QUALITY AND COST

- Real bugs found
- False findings on clean changes
- Rounds until review passes

KEEP THE LOOP BOUNDED

- After 3 rounds: reconsider
- Later passes: new diff only
- Fix bug classes; avoid repeats

Reported failure mode: one change went through 50+ review/fix rounds overnight.

> Failures become **reusable checks**; review knowledge **compounds**.

Source: Osmani, Lunt & Wang · 15 Oct 2026 · How the Claude Code team does code review · Team-reported practice.

Put the model back in the system



Inner ring: one call · Middle ring: across calls · Outer ring: engineering boundary

01	**LLM**	A probability generator: tokens in, a distribution over the next token out
02	**Prompt**	This call's conditions: system / user / few-shot
03	**Context**	Every token visible to this call; the window is finite
04	**Memory**	State beyond the window: write / retrieve / compress / forget
05	**Agent loop**	Observe → act → observe; state lives outside
06	**Harness**	Tools, sandbox, permissions, verification, logs

> The model **proposes**; the system manages context and state, runs tools, and **checks results**.

Acceleration: from basic use to gold medals

2023 · BEFORE

Basic use

- Explain concepts, search the literature, write code
- Do simple calculations, often wrong
- Complex proofs still need people
- Role: assistant

→ 2024–26

2025–26 · AFTER

Research level

- IMO 2025 gold level 35/42
- Lean proofs, machine-checkable
- OpenAI 2026: ≈3 Pro-hours of compute per result*
- Role: candidate generator

2024.07 **28/42** silver → 2025.07 **35/42** gold level 12 months, +7 points

* Compute equivalent, not elapsed time · OpenAI, 6 Oct 2026 · Company report.

> This is a staircase with **three steps**.

Timeline: three steps

2023–24

Tools: search + verification

2023.12
FunSearch
LLM + evaluator + evolutionary search; a new cap-set construction

2024.01
AlphaGeometry
Neuro-symbolic; IMO geometry 25/30

2024.07
AlphaProof
Lean + RL; IMO 2024 silver 28/42 (+AG2)

2024–25

Reasoning: RL + test-time compute

2024.09
OpenAI o1
RL teaches thinking; test-time compute

2025.01
DeepSeek-R1
RLVR; reasoning behavior emerges; open weights

2025.05
AlphaEvolve
Evolutionary coding agent; 4×4 complex matrix product in 48 multiplications

2025–26

Scale: from benchmarks to open problems

2025.07
IMO 2025
Gold level 35/42; natural language, no tools

2025
Formal gold (claimed)
Aristotle et al.; Lean-checkable

2026.05–10
OpenAI
Unit-distance counterexample; NS / Riemann-related (claimed); 372 problems (report)

> Every leap combines a **generator, a verifier and search**.

How they describe it

> Public material covers part of the method; the full training recipe stays private. Note the asymmetry: some results are easy to verify, hard to prove. Quotes below are **public statements**, excerpted or paraphrased.

DATE	SYSTEM / WORK	PUBLIC STATEMENT	READING
2023.12	**FunSearch**	"pairing a pre-trained LLM ... with an automated evaluator"	search
2024.01	**AlphaGeometry**	"100 million synthetic theorems and proofs"	synthetic data
2024.07	**AlphaProof**	autoformalization → Lean; RL + test-time RL	formalization
2024.09	**o1**	"spend more time thinking through problems"	reasoning
2025.01	**R1**	"directly apply RL ... rule-based rewards"	RLVR
2025.05	**AlphaEvolve**	"evolutionary coding agent"	evolve code
2025.07	**IMO 2025**	"natural language ... no tools"	end to end
2026.09–10	**OpenAI**	"NS solution; 372 problems; 722 papers"	open problems

> The keywords change; the underlying move stays: **propose, verify, search again**.

The common pattern: generate → verify → search

GENERATE

LLM

Candidate proofs, programs, constructions

→

VERIFY

Lean / tests / numerics

Returns right or wrong, a score, a counterexample

→

SEARCH / LEARN

best-of-N · trees · evolution · RL

Turns the signal into the next policy

↺ Back to the generator: the next round is stronger

MANUAL ONLY

Review stays manual; scale stalls

ONE SHOT

One draw per attempt; luck decides

BOTH

Probability is filtered into correctness, then written back into the model

> The model supplies probability; **the verifier supplies the gradient**.

Some guesses: how the big results may have happened

The following mechanism hypotheses come from public papers, blogs, and benchmarks, and are labeled as **guesses**.

01 DATA

Autoformalization + synthesis: informal problems → Lean; rejection sampling keeps the verifiable ones.

02 TRAINING

RLVR + long CoT: verifiable answers let rewards scale.

03 INFERENCE

Test-time search: best-of-N, tree search, evolution.

04 DISTILLATION

Write search back into the model: less search next time.

05 ENGINEERING

Harness: parallelism, caching, evals, replay.

06 THE BET

Verification: the harder your verifier, the faster your flywheel.

OpenAI reports ≈3 ChatGPT Pro-hours of compute per result · 6 Oct 2026.

A reported compute budget leaves the **system design open**.

What remains: from solving to doing mathematics

Already happening

CONTEST PROBLEMS
IMO gold level; natural language, timed

FORMAL PROOFS
Lean-checkable; machine verification

CONSTRUCTION SEARCH
Cap sets, matrix multiplication, scheduling

OPEN PROBLEMS
372 in the OpenAI report; humans still curate

Still open

CHOOSING A PROBLEM
Which question is worth asking

DEFINITIONS
New concepts, new structures

LONG HORIZON
A research plan spanning months

TASTE
Beauty, importance, growth

> Solving can be searched; **doing mathematics needs direction**.

The motor and the assembly line

A story Eric Schmidt tells — and the AI parallel.

When the electric motor was invented, factory owners did the obvious thing: remove the central steam engine, put a motor in its place, and change nothing else. Efficiency rose by about 30%.

Decades later, someone realized the motor could be small. Every machine could have its own. The factory could be designed around the flow of production itself; the power source no longer dictated the layout.

The assembly line was born. Efficiency rose by an order of magnitude.

30%

Standalone · Prompting Harder
Resets each session · No compounding

10×+

Assembly line · Context Engineering
Persistent assets & rules · Continuous compounding

The gap is **work designed to compound**.

> Models are rented; context and systems are owned. Do not merely prompt in isolation; turn domain rules, tools, and evals into persistent assets.

> The gain is in **layout and compounding**.

The algorithm engineer's dilemma

It arrived about a year ago. Mathematics is next.

"If the model can write the code, what is my job?"

BEFORE AGENTS

Write the algorithm

- Implement, debug, optimize
- Own the code
- Value = code shipped

→ ~1 year

WITH AGENTS

Direct the search

- Frame the problem
- Design the harness
- Verify and own the system

> The job moves from producing answers to **directing the search**.

From the chat box to the working directory

For my longer tasks, a working directory makes rules and results reusable.

MODE A · CONVERSATION

Web chat

Discuss and explore
Questions, explanations, uploaded material

Tools depend on configuration
Web interfaces can also offer tools and memory

MODE B · IN-EDITOR

IDE editor

Edit close to the code
Files, diagnostics and repository context

Assist or run an agent
Capabilities depend on tools and permissions

MODE C · TOOL-ENABLED LOOP

Repo & CLI Agent

Tools & isolated agents
Symplectic integrators · Series expansion · Reviewers

Autonomous verification
Hypothesis → Experiment → Conservation checks → git

- OpenCode
- Claude Code
- Codex CLI
- Cursor
- Markdown SOP Protocol

> Files are interfaces. Build **portable research assets** in local workspaces.

`Files and checks are reusable across tools; rerun evaluations when the model or environment changes.`

Two attempts

Same intention. Different governance: from abdication to coaching.

OPENCLAW · EARLY ATTEMPT

Mistaken for calculator · Hands-off

- Expected the agent to autonomously deliver an end-to-end paper.
- Early model was brittle; hallucinations compounded without evals.
- Complete human abdication, hoping for an unguided miracle.

My outcome: no completed paper; too little guidance and checking.

CODEX · GPT-5.3–5.5 · ISTART

Act as Coach · Verification loop

- Postdoc extension; I framed clear goals (WHAT) while AI drafted (HOW).
- Refused to be a line-by-line fixer; coached by flagging flawed steps.
- Caught absurd derivation: $Ax = b,\ b \neq 0 \rightarrow x = 0$ ✕
- After 3 rejections, GPT-6.1 Sol scanned deeper issues for final fixes.

Lesson: verification asymmetry — evaluating is easier than drafting.

> Do not act as a line-by-line fixer; act as a **coach defining standards and counterexamples**.

Low-hanging fruit & compounding assets

Drafting a paper is only the start; compounding assets are evals and taste.

01 · HUMAN AS COACH

Frame WHAT & bounds

Define clear objectives; let AI explore the HOW. Point out logical flaws rather than patching code by hand.

02 · SYSTEMATIC EVALS

Dynamical checks

Energy-error behavior, symplecticity residuals and convergence tests provide complementary evidence.

03 · COMPOUNDING ASSETS

Taste & eval suites

A single paper depreciates quickly. What compounds is your knowledge base, codebase SOPs, and formal scorers.

As drafting becomes cheaper, I invest more in **problem selection, reusable knowledge and evaluation**. These assets can accumulate across projects.

> The central question: which problems are **worth tackling together with machines**?

A young person's opportunity

Celestial mechanics is becoming a data field. The scarce skill is finding the signal.



Data

Gaia. Satellite tracking. Surveys. N-body simulations.

`More data than hands.`



Craft

Denoise. Model. Verify. Agents run the pipeline.

`The new leverage.`



Signal

Resonances. Anomalies. New orbits.

`The old questions.`

More data than hands. New tools. Old questions. **That is a young person's advantage.**

> The bottleneck is **signal**.

From the next token to the next theorem

`token` → `probability` → `candidate` → `verification` → **`theorem`**

Probability gives breadth

Verification gives correctness

People set the direction

> The next theorem still begins with the **next token**.